

```

#practical 1
#TANISHQ SIJARIA
#23070521159
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load California Housing Dataset
housing = fetch_california_housing(as_frame=True)

df = housing.frame

print(df.head())
print(df.shape)

# Features and Target
X = df.drop("MedHouseVal", axis=1)
y = df["MedHouseVal"]

# Normalize
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)

# Deep Feedforward Neural Network
model = Sequential([
    Dense(256, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(128, activation="relu"),
    Dense(64, activation="relu"),
    Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

model.summary()

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=1
)

# Graphs
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])
plt.title("Loss")
plt.legend(["Train", "Validation"])

plt.subplot(1,2,2)
plt.plot(history.history["mae"])
plt.plot(history.history["val_mae"])
plt.title("MAE")

```

```

plt.legend(["Train", "Validation"])

plt.show()

# Prediction
pred = model.predict(X_test).flatten()

# Metrics
mae = mean_absolute_error(y_test, pred)
mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)

# Comparison
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": pred
})

print(results.head(20))

# Scatter Plot
plt.figure(figsize=(7,7))
plt.scatter(y_test, pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted House Prices")
plt.grid(True)
plt.show()

# Deeper Model
deep_model = Sequential([
    Dense(512, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(256, activation="relu"),
    Dense(128, activation="relu"),
    Dense(64, activation="relu"),
    Dense(32, activation="relu"),
    Dense(1)
])

deep_model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

history2 = deep_model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=1
)

```



```

MedInc HouseAge AveRooms AveBedrms Population AveOccup Latitude \
0 8.3252 41.0 6.984127 1.023810 322.0 2.555556 37.88
1 8.3014 21.0 6.238137 0.971880 2401.0 2.109842 37.86
2 7.2574 52.0 8.288136 1.073446 496.0 2.802260 37.85
3 5.6431 52.0 5.817352 1.073059 558.0 2.547945 37.85
4 3.8462 52.0 6.281853 1.081081 565.0 2.181467 37.85

```

```

Longitude MedHouseVal
0 -122.23 4.526
1 -122.22 3.585
2 -122.24 3.521
3 -122.25 3.413
4 -122.25 3.422

```

```
(20640, 9)
```

```
(16512, 8)
```

```
(4128, 8)
```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	2,304
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 1)	65

```
Total params: 43,521 (170.00 KB)
```

```
Trainable params: 43,521 (170.00 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/100
```

```
413/413 ————— 4s 5ms/step - loss: 0.5889 - mae: 0.5348 - val_loss: 0.3986 - val_mae: 0.4547
```

```
Epoch 2/100
```

```
413/413 ————— 2s 4ms/step - loss: 0.3748 - mae: 0.4309 - val_loss: 0.3723 - val_mae: 0.4213
```

```
Epoch 3/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.3586 - mae: 0.4130 - val_loss: 0.3441 - val_mae: 0.4061
```

```
Epoch 4/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.3271 - mae: 0.3987 - val_loss: 0.3750 - val_mae: 0.4041
```

```
Epoch 5/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.3312 - mae: 0.3893 - val_loss: 0.3586 - val_mae: 0.4148
```

```
Epoch 6/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.3091 - mae: 0.3844 - val_loss: 0.3410 - val_mae: 0.3961
```

```
Epoch 7/100
```

```
413/413 ————— 2s 4ms/step - loss: 0.2933 - mae: 0.3712 - val_loss: 0.3196 - val_mae: 0.3849
```

```
Epoch 8/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.2929 - mae: 0.3698 - val_loss: 0.3060 - val_mae: 0.3761
```

```
Epoch 9/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.2854 - mae: 0.3658 - val_loss: 0.3054 - val_mae: 0.3710
```

```
Epoch 10/100
```

```
413/413 ————— 2s 4ms/step - loss: 0.2824 - mae: 0.3631 - val_loss: 0.3144 - val_mae: 0.3967
```

```
Epoch 11/100
```

```
413/413 ————— 2s 4ms/step - loss: 0.2777 - mae: 0.3593 - val_loss: 0.2970 - val_mae: 0.3654
```

```
Epoch 12/100
```

```
413/413 ————— 2s 3ms/step - loss: 0.2756 - mae: 0.3567 - val_loss: 0.3066 - val_mae: 0.3777
```

```
Epoch 13/100
```

```
413/413 ————— 1s 3ms/step - loss: 0.2696 - mae: 0.3521 - val_loss: 0.3192 - val_mae: 0.3836
```